



Proyecto 1 - Uso de un protocolo existente

Redes

Giovanni Santos – 22523

Especificación de los servidores MCP desarrollados.

El servidor uvg-hr-local-server fue desarrollado en Python utilizando una base de datos SQLite y se comunica a través del estándar JSON-RPC 2.0. De acuerdo con la implementación, se exponen tres endpoints principales que permiten la integración del agente con el subsistema de Recursos Humanos:

- **consultar_directorio:** Herramienta de solo lectura que permite buscar miembros del personal corporativo por nombre, departamento o correo electrónico.
 - **Parámetro:** query (String, requerido). Debe tener entre 2 y 100 caracteres.
 - **Retorno:** Un arreglo de empleados que incluye employee_id, nombre completo, departamento, puesto, correo, teléfono y estado laboral.
- **consultar_vacaciones:** Herramienta de solo lectura que calcula el saldo de días de vacaciones de un empleado para un año calendario.
 - **Parámetros:** employee_id (String, requerido, ej. EMP-001) y year (Integer, requerido, entre 2000 y 2100).
 - **Retorno:** Objeto JSON con los días a los que tiene derecho, los días utilizados y los días disponibles.
- **registrar_solicitud_permiso:** Herramienta de escritura que crea una solicitud de ausencia validando la identidad y la coherencia de las fechas.
 - **Parámetros:** employee_id (String, requerido), start_date (String, requerido, formato ISO YYYY-MM-DD), end_date (String, requerido, formato ISO YYYY-MM-DD) y reason (String, requerido, máximo 300 caracteres).
 - **Retorno:** Genera un request_id, confirmando el estado "pendiente" y la marca de tiempo de la creación.

Análisis de la comunicación

El intercambio de mensajes MCP muestra el siguiente comportamiento en el modelo de red, según Wireshark:

- **Capa de Enlace:** Dado que la captura se realizó sobre tráfico local debido a las restricciones de seguridad al hacer el deploy en Render, se observa el uso de la interfaz de loopback (lo0). Las tramas se encapsulan mediante Null/Loopback (id 0) en lugar de utilizar direcciones MAC con el protocolo Ethernet tradicional.
- **Capa de Red:** Se emplea el Protocolo de Internet versión 4 (IPv4). La dirección IP de origen y destino es 127.0.0.1 (localhost). Esto confirma que el cliente y el servidor se están comunicando dentro de la misma máquina en esta captura.
- **Capa de Transporte:** La comunicación se establece de forma confiable utilizando el Protocolo de Control de Transmisión (TCP). El paquete 393 muestra que el segmento viaja desde el puerto efímero de origen 55744 hacia el puerto de destino 8000 (puerto de escucha del servidor).

- **Capa de Aplicación:** Se observa el uso de HTTP/1.1 para transportar los mensajes. Específicamente, se captura un método POST dirigido al endpoint /messages?sessionId=0485e27d... con el tipo de contenido application/json. A nivel de datos de aplicación, viaja un mensaje JSON-RPC que corresponde al mensaje de sincronización (petición). El payload decodificado revela: {"jsonrpc": "2.0", "id": 1, "method": "initialize", "params": {"protocolVersion": "2025-11-25", "capabilities": {}, "clientInfo": {"name": "chatbot", "version": "0.1.0"}}}

Capturas Wireshark:

No.	Time	Source	Destination	Protocol	Length	Info
382	01:47:25.077929	127.0.0.1	127.0.0.1	HTTP	210	GET /sse HTTP/1.1
393	01:47:25.181042	127.0.0.1	127.0.0.1	HTTP	227	POST /messages?sessionId=0485e27d-b717-409c-a885-e89aea50719f HTTP/1.1, JSON (application/json)
396	01:47:25.182475	127.0.0.1	127.0.0.1	HTTP	154	HTTP/1.1 202 Accepted
409	01:47:25.184159	127.0.0.1	127.0.0.1	HTTP	127	POST /messages?sessionId=0485e27d-b717-409c-a885-e89aea50719f HTTP/1.1, JSON (application/json)
412	01:47:25.184629	127.0.0.1	127.0.0.1	HTTP	154	HTTP/1.1 202 Accepted
423	01:47:25.185957	127.0.0.1	127.0.0.1	HTTP	121	POST /messages?sessionId=0485e27d-b717-409c-a885-e89aea50719f HTTP/1.1, JSON (application/json)
426	01:47:25.186362	127.0.0.1	127.0.0.1	HTTP	154	HTTP/1.1 202 Accepted
438	01:47:25.187969	127.0.0.1	127.0.0.1	HTTP	182	POST /messages?sessionId=0485e27d-b717-409c-a885-e89aea50719f HTTP/1.1, JSON (application/json)
442	01:47:25.188894	127.0.0.1	127.0.0.1	HTTP	154	HTTP/1.1 202 Accepted
454	01:47:25.190171	127.0.0.1	127.0.0.1	HTTP	206	POST /messages?sessionId=0485e27d-b717-409c-a885-e89aea50719f HTTP/1.1, JSON (application/json)
458	01:47:25.190832	127.0.0.1	127.0.0.1	HTTP	154	HTTP/1.1 202 Accepted
471	01:47:25.192130	127.0.0.1	127.0.0.1	HTTP	282	POST /messages?sessionId=0485e27d-b717-409c-a885-e89aea50719f HTTP/1.1, JSON (application/json)
474	01:47:25.193841	127.0.0.1	127.0.0.1	HTTP	154	HTTP/1.1 202 Accepted

Wireshark - Packet 393 - Loopback: lo0

```

> Frame 393: Packet, 227 bytes on wire (1816 bits), 227 bytes captured (1816 bits) on interface lo0, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 55744, Dst Port: 8000, Seq: 261, Ack: 1, Len: 171
> [2 Reassembled TCP Segments (431 bytes): #392(260), #393(171)]
> Hypertext Transfer Protocol
  JavaScript Object Notation: application/json
    Object
      Member: jsonrpc
      Member: id
      Member: method
        [Path with value: /method:initialize]
        [Member with value: method:initialize]
        String value: initialize
        Key: method
        [Path: /method]
      Member: params
        Object
          Member: protocolVersion
          Member: capabilities
          Member: clientInfo
        Key: params
        [Path: /params]
  
```

```

0000 50 4f 53 54 20 2f 6d 65 73 73 61 67 65 73 3f 73 POST /me ssages?s
0010 65 73 73 69 6f 6e 49 64 3d 30 34 38 35 65 32 37 sessionId =0485e27
0020 64 2d 62 37 31 37 2d 34 30 39 63 2d 61 38 38 35 d-b717-4 09c-a885
0030 2d 65 38 39 61 65 61 35 30 37 31 39 66 20 48 54 -e89aea5 0719f HT
0040 54 50 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 6c 6f TP/1.1· Host: lo
0050 63 61 6c 68 6f 73 74 3a 38 30 30 30 0d 0a 55 73 calhost: 8000·Us
0060 65 72 2d 41 67 65 6e 74 3a 20 70 79 74 68 6f 6e er-Agent : python
0070 2d 72 65 71 75 65 73 74 73 2f 32 2e 33 34 2e 32 -request s/2.34.2
0080 0d 0a 41 63 63 65 70 74 2d 45 6e 63 6f 64 69 6e ··Accept -Encodin
0090 67 3a 20 67 74 69 70 2c 20 64 65 66 6c 61 74 65 g: gzip, deflate
00a0 2c 20 7a 73 74 64 0d 0a 41 63 63 65 70 74 3a 20 , zstd·· Accept:
00b0 2a 2f 2a 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a */*··Con nection:
00c0 20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a 43 6f 6e keep-al ive··Con
00d0 74 65 6e 74 2d 4c 65 6e 67 74 68 3a 20 31 37 31 tent-Len gth: 171
00e0 0d 0a 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a 20 ··Conten t-Type:
00f0 61 70 70 6c 69 63 61 74 69 6f 6e 2f 6a 73 6f 6e applica tion/json
0100 0d 0a 0d 0a 7b 22 6a 73 6f 6e 72 70 63 22 3a 20 ···{"js onrpc":
0110 22 32 2e 30 22 2c 20 22 69 64 22 3a 20 31 2c 20 "2.0", " id": 1,
0120 22 6d 65 74 68 6f 64 22 3a 20 22 69 6e 69 74 69 "method" : "initi
0130 61 6c 69 7a 65 22 2c 20 22 70 61 72 61 6d 73 22 alize", "params"
0140 3a 20 7b 22 70 72 6f 74 6f 63 6f 6c 56 65 72 73 : {"prot ocolVers
0150 69 6f 6e 22 3a 20 22 32 30 32 35 2d 31 31 2d 32 ion": "2 025-11-2
0160 35 22 2c 20 22 63 61 70 61 62 69 6c 69 74 69 65 5", "cap abilitie
0170 73 22 3a 20 7b 7d 2c 20 22 63 6c 69 65 6e 74 49 s": {}, "clientI
0180 6e 66 6f 22 3a 20 7b 22 6e 61 6d 65 22 3a 20 22 nfo": {" name": "
0190 63 68 61 74 62 6f 74 22 2c 20 22 76 65 72 73 69 chatbot", "versi
01a0 6f 6e 22 3a 20 22 30 2e 31 2e 30 22 7d 7d 7d on": "0. 1.0"}}}
  
```

Packet (227 bytes) Reassembled TCP (431 bytes)

Conclusiones y comentarios

La implementación del Model Context Protocol (MCP) proporciona una solución abierta y estandarizada que supera la limitación inherente de los LLMs de operar exclusivamente sobre su base de conocimiento aislada. Al estructurar las herramientas bajo un formato JSON-RPC agnóstico, se logró integrar exitosamente la API de Google Gemini con un sistema de archivos, un entorno Git y una base de datos relacional sin depender de SDKs propietarios.

El desarrollo del servidor local orientado a Recursos Humanos demuestra la viabilidad de utilizar asistentes conversacionales para democratizar el acceso a los datos. Permite que usuarios sin conocimientos técnicos en lenguajes de consulta como SQL puedan interactuar con bases de datos transaccionales de forma natural e intuitiva. La implementación manual del protocolo fortalece la comprensión profunda de la arquitectura cliente-servidor, validando que el enrutamiento, la validación de esquemas y la gestión del estado se pueden orquestar eficientemente desde la terminal.